

AI Usage Log : Sprint #1 Mamadou Camara

Task 1.1: Define the directory structure

Purpose of AI Use: Brainstorming and proposing the best structure to avoid merge conflicts

Chat Link or Prompt/Response:

Prompt: What's the best structure to avoid merge conflicts, because for example, if I would work on the authentication and add a schema while my teammates also adds his schemas it would create a conflict.

Response: The conflict you're describing comes from having one shared `models.py` and `schemas.py` — every feature owner has to edit the same file, so Git sees overlapping line changes even when your logic doesn't actually overlap. The fix is to split those files per-feature too, the same way you split routers.

Structure that isolates each person's work

backend/

```
|— app.py
|— database.py
|— security.py
|— requirements.txt
|— models/
|   |— _init_.py  ← imports everything so app.py sees one namespace
|   |— user.py    ← your model
|   |— resume.py  ← teammate's model
|— schemas/
|   |— _init_.py
|   |— auth.py    ← your Pydantic schemas
|   |— resume.py  ← teammate's Pydantic schemas
|— routers/
|   |— _init_.py
|   |— auth.py
|   |— resume.py
```

Why this matters more than it looks

Git merges line-by-line — two people can edit the *same file* in *different functions* without a real conflict, but Git often still flags it if the edits are close together or one person reformats. Splitting by feature means your changes and your teammate's changes usually never touch the same file at all, so Git auto-merges cleanly without a conflict prompt.

AI-Suggested Content: Generated an example of a structure..

Validation: Compared the practices of various projects against the prompt answer and reviewed them during the team meeting.

Decision: Accepted.

Reflection: AI helped identify the safest way of organizing the directory to prevent any merge conflicts

Responsible Person: Mamadou Camara

Task 1.2: Creating setup/running files for the application

Purpose of AI Use: Proposing and create summary files to limit the number of commands needed to run the application.

Chat Link or Prompt/Response:

Prompt: Show me the setup/running files needed to be able to use a single command to do both of these action.

Response:

setup.sh

```
#!/bin/bash
```

```
# One-time setup for CareerConnect Sprint 1.
```

```
# Run this once after cloning the repo (or after pulling changes that touch
```

```
# requirements.txt / package.json).
```

```
#
```

```
# Usage:
```

```
# cd "Sprint 1"
```

```
# chmod +x setup.sh (only needed the very first time)
```

```
# ./setup.sh
```

```
set -e # stop immediately if any command fails, instead of plowing ahead
```

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
```

```
BACKEND_DIR="$SCRIPT_DIR/backend"
```

```
FRONTEND_DIR="$SCRIPT_DIR/frontend"
```

```
echo "=== Setting up backend ==="
```

```
cd "$BACKEND_DIR"
```

```
if [ ! -d "venv" ]; then
```

```
  echo "Creating virtual environment..."
```

```
  python3 -m venv venv
```

```
fi
```

```
source venv/bin/activate
```

```
pip install --upgrade pip > /dev/null
```

```
pip install -r requirements.txt
```

```
if [ ! -f ".env" ]; then
```

```
  echo ""
```

```
  echo "No .env file found — copying .env.example to .env"
```

```
  cp .env.example .env
```

```
  echo "IMPORTANT: open backend/.env and fill in the real DATABASE_URL"
```

```
  echo "(ask a teammate for the shared Supabase connection string)."
```

```
  echo ""
```

```
fi
```

```
deactivate
```

```
echo "=== Setting up frontend ==="
```

```
cd "$FRONTEND_DIR"
```

```
npm install
```

```
echo ""
```

```

echo "Setup complete."

if ! grep -q "postgresql://postgres" "$BACKEND_DIR/.env" 2>/dev/null || grep -q "YOUR-SHARED-PASSWORD\|xxxxxxxxxxxxx"
"$BACKEND_DIR/.env" 2>/dev/null; then

    echo "REMINDER: backend/.env still needs a real DATABASE_URL before you can run the app."

fi

echo "Run './run.sh' from the Sprint 1 folder to start both servers."

```

run.sh

```

#!/bin/bash

# Runs both the backend and frontend dev servers together, in one terminal.

# Assumes you've already run setup.sh at least once.

#

# Usage:

# cd "Sprint 1"

# chmod +x run.sh (only needed the very first time)

# ./run.sh

#

# Press Ctrl+C once to stop BOTH servers — normally you'd need to do this

# separately in two terminal tabs; this script handles that for you.


SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

BACKEND_DIR="$SCRIPT_DIR/backend"

FRONTEND_DIR="$SCRIPT_DIR/frontend"


# If anything goes wrong, or the user hits Ctrl+C, kill both background

# processes rather than leaving one orphaned and still holding its port.

cleanup() {

    echo ""

    echo "Stopping servers..."

    kill "$BACKEND_PID" "$FRONTEND_PID" 2>/dev/null

    exit 0

```

```

}

trap cleanup INT TERM

echo "=== Starting backend (http://localhost:5000) ==="

cd "$BACKEND_DIR"

source venv/bin/activate

uvicorn app:app --reload --port 5000 &

BACKEND_PID=$!

deactivate


echo "=== Starting frontend ==="

cd "$FRONTEND_DIR"

npm run dev &

FRONTEND_PID=$!


echo ""

echo "Both servers running. Press Ctrl+C to stop both."


# Wait for either process to exit (or for Ctrl+C to trigger cleanup above)

wait

```

How to call them (per the usage comments above, matching your README style):

```

bash

cd "Sprint 1"

chmod +x setup.sh run.sh # only needed the very first time

./setup.sh           # one-time, or after pulling changes to requirements.txt/package.json

./run.sh             # starts both servers; Ctrl+C stops both

```

AI-Suggested Content: Generated the sh files and the way to run them..

Validation: Compared the common practices of various projects against the prompt answer and reviewed them during the team meeting.

Decision: Accepted but modified.

Reflection: AI helped create scripts to make the setup and running process less tedious. However, I modified it to fit mac/windows systems more easily.

Responsible Person: Mamadou Camara

Task 1.3: Commit description template

Purpose of AI Use: Proposing and creating a descriptive commit description that includes a summary, Files changed and a checklist of test plans.

Chat Link or Prompt/Response:

Prompt: Give me a descriptive commit message for these changes ui + email check and a description including check boxes of tests done

Answer:

PR / commit description with a test checklist (check off what you actually verified — I've marked the ones I could confirm from code, not from clicking through the real app):

markdown

Summary

- Auth input fields now have an explicit white background and black text
- Password fields use real eye / eye-off SVG icons instead of "Yes"/"No" text
- Removed the Google icon from both Google sign-in buttons (text-only now)
- Added email format validation (regex-based) on both Login and Register, shown as an inline error message; blocks submit if invalid

Files changed

- ` Sprint 1/frontend/src/components/Login.tsx`
- ` Sprint 1/frontend/src/components/Register.tsx`
- ` Sprint 1/frontend/src/styles/auth.css`

Test plan

- [] `npm run dev` starts with no errors
- [] Login page renders with white input backgrounds and black input text
- [] Register page renders with white input backgrounds and black input text
- [] Password eye icon toggles between hidden/visible text on Login

- [] Password eye icon toggles between hidden/visible text on Register
- [] "Sign in with Google" button shows no icon/logo
- [] "Sign up with Google" button shows no icon/logo
- [] Typing an invalid email (e.g. `test`, `test@`, `test@domain`) and leaving the field shows the inline error
- [] Error message disappears once a valid email format is typed
- [] Submitting the form with an invalid email is blocked (no request sent)
- [] Submitting with a valid email + correct password still logs in successfully
- [] Submitting with a valid email + correct fields still registers successfully
- [] `npx tsc --noEmit` passes with no type errors

AI-Suggested Content: Generated a markdown file containing the requested elements

Validation: Compared the industry process and thought about what exactly I deem necessary.

Decision: Accepted but modified to remove unnecessary information and the files changes.

Reflection: AI helped create a very good description of the commit and made the writing process faster. However, I modified it because it had more information than needed.

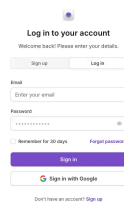
Responsible Person: Mamadou Camara

Task 1.4: Frontend coding of the UI for registering/log-in

Purpose of AI Use: Proposing and creating a template of the react code necessary to achieve a specific UI esthetic.

Chat Link or Prompt/Response:

Prompt: How would you update the login/register react code to fit a modern esthetic. I want to use blueish tones and for the login/register to be next to each other with buttons. Take as inspiration this



style :

Answer: Check Appendix

AI-Suggested Content: Generated a updated react files containing the requested changes

Validation: Compared the old code with the suggestion and looked through react documentations.

Decision: Accepted but modified to update the show/hide password logic and to update certain UI elements.

Reflection: AI helped create a very template that fits the demanded look. However, some of the chosen colors / features were not intuitive for the user so I had to verify everything to ensure correct code.

Responsible Person: Mamadou Camara

Task 1.5: Handling password security

Purpose of AI Use: Brainstorming and proposing the best process to handle password security for this project.

Chat Link or Prompt/Response:

Prompt: What's the best structure to avoid simply storing passwords in plain text. I.e using hashing, etc.

Response: The conflict you're describing comes from having one shared `models.py` and `schemas.py` — every feature owner has to edit the same file, so Git sees overlapping line changes even when your logic doesn't actually overlap. The fix is to split those files per-feature too, the same way you split routers.

Storage (`security.py`, `hash_password`)

- A password is never stored as plain text. `bcrypt.hashpw()` hashes it with a random salt (`bcrypt.gensalt()`) baked in automatically, and only the resulting hash goes into `password_hash` on the `User` row.
- `bcrypt` is a deliberately slow, adaptive hashing algorithm (unlike a fast hash like SHA-256/MD5), which makes brute-forcing a leaked hash database computationally expensive.

Verification (`verify_password`, used in `POST /api/login`)

- On login, the plaintext password from the request is hashed-and-compared against the stored hash via `bcrypt.checkpw()` — it never reverses the hash, since `bcrypt` is one-way by design.
- A failed login returns a generic `401 "Invalid email or password"` — it doesn't say *which* part was wrong, so an attacker can't use the error to discover whether an email is registered.

Session token (`create_access_token` / `decode_access_token`)

- After a successful register/login, the server issues a signed JWT (`HS256`, `SECRET_KEY`) containing the user's id and a 24-hour expiry — this is what proves "I already logged in" on later requests, without the server needing to store sessions.

Is this best practice?

What's solid:

- ☒ `bcrypt` for hashing, never plaintext or a fast hash — this is the correct baseline.
- ☒ Salting is automatic and per-password (`bcrypt`'s built-in behavior) — protects against rainbow-table attacks.
- ☒ Generic error message on login failure — avoids leaking which emails are registered (user enumeration).
- ☒ Passwords never appear in API responses (`UserOut` excludes `password_hash`).

AI-Suggested Content: Generated an example of a structure to store and verify passwords.

Validation: Compared the practices of others against the prompt answer and reviewed them during the team meeting.

Decision: Accepted.

Reflection: AI helped identify the safest way of organizing password handling

Responsible Person: Mamadou Camara

Task 1.6: Adapting the calling logic to fit the new Updated Dashboard UI

Purpose of AI Use: Pointing out the changes needed to fully connect the backend resume logic to the updated dashboard

Chat Link or Prompt/Response:

Prompt: My teammate has updated the dashboard UI. Analyze in-depth the changes. What are the missing calls needed to be added for the backend to be integrated to the dashboard.

Response:

1. Pass the auth token into **Dashboard**, since every resume call needs it (**App.tsx** already has it in **localStorage/sessionStorage**):

tsx

```
interface DashboardProps {  
  user: User;  
  
  token: string; // ← new  
  
  onLogout: () => void;  
}
```

and in **App.tsx**, pass it: `<Dashboard user={user} token={token} onLogout={handleLogout} />` (token already exists there as the value read from storage).

2. Drop the local **Resume** interface — import the one from **api.ts** instead, so the two never drift apart (**api.ts**'s version already has the correct **user_id** field the local one is missing):

tsx

```
import { uploadResume, listMyResumes, deleteResume, downloadResume, type Resume } from "../api";
```

3. Fetch existing resumes on mount:

tsx

```
useEffect(() => {  
  
  listMyResumes(token)  
  
    .then(setResumes)  
  
    .catch(() => setError("Couldn't load your resumes."));  
}
```

```
}, [token]);
```

4. Replace the fake local insert in **addFile** with a real upload:

tsx

```
async function addFile(file: File | undefined) {  
  
  if (!file) return;  
  
  const ext = file.name.split(".").pop()?.toLowerCase() ?? "";  
  
  if (!ALLOWED_TYPES.includes(ext)) {  
  
    setError("Only PDF or DOCX files can be uploaded.");  
  
    return;  
  
  }  
  
  if (file.size > MAX_SIZE_MB * 1024 * 1024) {  
  
    setError(`That file is over ${MAX_SIZE_MB} MB. Try a smaller version.`);  
  
    return;  
  
  }  
  
  setError("");  
  
  try {  
  
    const { resume } = await uploadResume(file, token);  
  
    setResumes((prev) => [resume, ...prev]);  
  
  } catch (err) {  
  
    setError(err instanceof Error ? err.message : "Upload failed");  
  
  }  
  
}
```

(This also means **addFile** needs to become **async**, and its callers (**handleDrop**, the file input's **onChange**) are already fine calling an async function without awaiting it, but consider a **uploading** boolean state to disable the drop zone/button mid-upload.)

5. Add delete/download actions to each card (backend already supports both):

tsx

```
<li key={r.id} className="cc-card">  
  
  { /* ...existing thumb/name/meta... */ }  
  
  <div className="cc-card-actions">  
  
    <button type="button" onClick={() => downloadResume(r.id, r.filename, token)}>Download</button>  
  
    <button
```

```

    type="button"

    onClick={async () => {

      await deleteResume(r.id, token);

      setResumes((prev) => prev.filter((x) => x.id !== r.id));

    }}
  >
    Delete
  </button>
</div>
</li>

```

AI-Suggested Content: Generated the additional calls needed to connect the backend.

Validation: Compared the logic of the login/register code which does the same process.

Decision: Accepted.

Reflection: AI helped me quickly add everything missing.

Responsible Person: Mamadou Camara

Appendix

Task 1.4 Prompt Result:

Here's the redesigned code, matching your screenshot's layout with a blue palette instead of purple. I'm giving you a shared stylesheet plus both components — heavily commented so you can review every decision before applying anything yourself.

1. src/styles/auth.css (new file — shared by both pages)

css

```

/* =====

Shared styling for the Login and Register pages.

Both components import this same file so the "look" (colors,
spacing, card shape) stays identical between the two screens —
only the form fields inside differ.

===== */

```

```

/* CSS custom properties (variables) — change these to re-theme

```

the entire auth UI from one place instead of hunting through

every rule below. */

```
.root {  
  
  --auth-primary: #2563eb;    /* main blue — buttons, active tab, links */  
  
  --auth-primary-hover: #1d4ed8; /* darker blue for hover states */  
  
  --auth-primary-light: #eff6ff; /* very light blue — page background */  
  
  --auth-border: #e2e8f0;    /* neutral border color for inputs/cards */  
  
  --auth-text: #0f172a;      /* near-black for headings */  
  
  --auth-text-muted: #64748b; /* gray for subtitles/help text */  
  
  --auth-radius: 10px;      /* corner rounding used everywhere */  
  
}
```

/* Full-page container: centers the card both horizontally and

vertically, and gives the page its pale blue background

(matches the light, airy feel of the screenshot). */

```
.auth-page {  
  
  min-height: 100vh;  
  
  display: flex;  
  
  align-items: center;  
  
  justify-content: center;  
  
  background-color: var(--auth-primary-light);  
  
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;  
  
  padding: 24px; /* keeps the card off the screen edges on mobile */  
  
}
```

/* The white card itself. */

```
.auth-card {  
  
  width: 100%;  
  
  max-width: 360px;  
  
  background: #ffffff;  
  
  border-radius: 16px;  
  
  box-shadow: 0 4px 24px rgba(15, 23, 42, 0.08); /* soft drop shadow */  
  
  padding: 32px;  
  
}
```

```
/* The little rounded icon square at the top of the card. */  
  
.auth-icon {  
  
  width: 48px;  
  
  height: 48px;  
  
  margin: 0 auto 20px auto; /* centers it, adds space below */  
  
  border-radius: 12px;  
  
  background: linear-gradient(135deg, #dbeafe, #bfdbfef); /* light blue gradient */  
  
  display: flex;  
  
  align-items: center;  
  
  justify-content: center;  
  
}
```

```
/* The dot inside the icon square (stand-in for a logo/avatar). */  
  
.auth-icon-dot {  
  
  width: 14px;  
  
  height: 14px;  
  
  border-radius: 50%;  
  
  background: var(--auth-primary);  
  
}
```

```
.auth-title {  
  
  
  
  text-align: center;  
  
  font-size: 20px;  
  
  font-weight: 700;  
  
  color: var(--auth-text);  
  
  margin: 0 0 6px 0;  
  
}
```

```
.auth-subtitle {  
  
  
  
  text-align: center;  
  
  font-size: 14px;  
  
  color: var(--auth-text-muted);  
  
  margin: 0 0 24px 0;  
  
}
```

/* The "Sign up / Log in" pill-shaped tab switcher. */

```
.auth-tabs {  
  
  display: flex;  
  
  background: #f1f5f9; /* light gray track behind the pills */  
  
  border-radius: var(--auth-radius);  
  
  padding: 4px;  
  
  margin-bottom: 24px;  
  
}
```

/* Each tab is a <Link>, styled to look like a button, not a hyperlink. */

```
.auth-tab {  
  
  flex: 1;  
  
  text-align: center;  
  
  padding: 8px 0;  
  
  border-radius: 6px;  
  
  font-size: 14px;  
  
  font-weight: 600;  
  
  color: var(--auth-text-muted);  
  
  
  text-decoration: none; /* removes the default underline from <Link> */  
  
  transition: background 0.15s ease, color 0.15s ease;  
  
}
```

/* The currently active tab (the page you're actually on) gets a
white background so it "pops" out of the gray track — this class
is applied conditionally in the JSX based on which page we're on. */

```
.auth-tab.active {  
  
  background: #ffffff;  
  
  color: var(--auth-text);  
  
  box-shadow: 0 1px 3px rgba(15, 23, 42, 0.1);  
  
}
```

```
.auth-field {  
  
  margin-bottom: 16px;  
  
}
```

```
.auth-label {  
  
  display: block;  
  
  font-size: 13px;  
  
  font-weight: 600;  
  
  color: var(--auth-text);  
  
  margin-bottom: 6px;  
  
}
```

```
.auth-input {  
  
  width: 100%;  
  
  padding: 10px 12px;  
  
  font-size: 14px;  
  
  border: 1px solid var(--auth-border);  
  
  border-radius: 8px;  
  
  outline: none;
```

```
  
  box-sizing: border-box; /* so padding doesn't push the input wider than its container */  
  
  transition: border-color 0.15s ease;  
  
}
```

```
.auth-input:focus {  
  
  border-color: var(--auth-primary); /* blue outline on focus instead of the default browser glow */  
  
}
```

```
/* Wraps the password input + the show/hide eye button so the  
   button can be positioned inside the input's right edge. */
```

```
.auth-password-wrapper {  
  
  position: relative;  
  
}
```

```
.auth-password-wrapper .auth-input {  
  
  padding-right: 40px; /* leaves room so typed text doesn't run under the eye icon */  
  
}
```

```
.auth-eye-btn {  
  
  position: absolute;
```

```
right: 10px;

top: 50%;

transform: translateY(-50%);

background: none;

border: none;

cursor: pointer;

color: var(--auth-text-muted);

font-size: 16px;

padding: 0;

line-height: 1;

}
```

```
/* Row holding "Remember me" checkbox + "Forgot password" link
```

```
(only shown on the Login page, not Register). */
```

```
.auth-row {

display: flex;

align-items: center;

justify-content: space-between;

margin-bottom: 20px;

font-size: 13px;

}
```

```
.auth-remember {

display: flex;

align-items: center;

gap: 8px;

color: var(--auth-text);

}
```

```
.auth-forgot-link {

color: var(--auth-primary);

font-weight: 600;

text-decoration: none;

cursor: pointer;

background: none;

border: none;

font-size: 13px;
```

```
}
```

```
/* The main colored submit button (Sign in / Sign up). */
```

```
.auth-submit-btn {
```

```
width: 100%;
```

```
padding: 11px 0;
```

```
background: var(--auth-primary);
```

```
color: #ffffff;
```

```
border: none;
```

```
border-radius: 8px;
```

```
font-size: 14px;
```

```
font-weight: 600;
```

```
cursor: pointer;
```

```
transition: background 0.15s ease;
```

```
}
```

```
.auth-submit-btn:hover:not(:disabled) {
```

```
background: var(--auth-primary-hover);
```

```
}
```

```
.auth-submit-btn:disabled {
```

```
opacity: 0.6;
```

```
cursor: not-allowed;
```

```
}
```

```
/* The secondary "Sign in with Google" outline button. */
```

```
.auth-google-btn {
```

```
width: 100%;
```

```
padding: 10px 0;
```

```
margin-top: 10px;
```

```
background: #ffffff;
```

```
color: var(--auth-text);
```

```
border: 1px solid var(--auth-border);
```

```
border-radius: 8px;
```

```
font-size: 14px;
```

```
font-weight: 600;
```

```
cursor: pointer;

display: flex;

align-items: center;
```

```
justify-content: center;

gap: 8px;

}
```

```
.auth-google-btn:hover {

background: #f8fafc;

}
```

```
.auth-bottom-text {

text-align: center;

font-size: 13px;

color: var(--auth-text-muted);

margin-top: 20px;

}
```

```
.auth-bottom-text a {

color: var(--auth-primary);

font-weight: 600;

text-decoration: none;

}
```

```
.auth-error {

background: #fef2f2;

color: #dc2626;

border: 1px solid #fecaca;

border-radius: 8px;

padding: 10px 12px;

font-size: 13px;

margin-bottom: 16px;

}
```

2. src/components/Login.jsx (rewritten)

```
jsx
```

```
import { useState } from "react";
```

```

import { Link, useNavigate } from "react-router-dom";

import { loginUser } from "../api";

import "../styles/auth.css";

export default function Login({ onSuccess }) {

  // form field state — same pattern as before, just email/password

  const [form, setForm] = useState({ email: "", password: "" });


  // UI-only state additions to match the new design:

  const [showPassword, setShowPassword] = useState(false); // toggles the eye icon

  const [remember, setRemember] = useState(false); // "Remember for 30 days" checkbox

  const [error, setError] = useState("");

  const [loading, setLoading] = useState(false);


  const navigate = useNavigate(); // lets us redirect after a successful login


  const handleChange = (e) => {

    setForm({ ...form, [e.target.name]: e.target.value });

  };


  const handleSubmit = async (e) => {

    e.preventDefault();

    setError("");

    setLoading(true);

    try {

      const data = await loginUser(form);


      // "Remember for 30 days" implemented for real (not just cosmetic):

      // - checked -> localStorage: token survives closing the browser tab

      // - unchecked -> sessionStorage: token is cleared when the tab closes

      // NOTE: your JWT itself still expires after 24h server-side


      // (see ACCESS_TOKEN_EXPIRE_HOURS in security.py) — this only controls

      // how long the BROWSER holds onto it, not how long the token is valid.

      // To actually honor "30 days" you'd extend that backend expiry too.

      const storage = remember ? localStorage : sessionStorage;

```

```

storage.setItem("token", data.token);

onSuccess && onSuccess(data.user);

navigate("/dashboard"); // adjust to whatever your protected route is named
} catch (err) {

  setError(err.message);

} finally {

  setLoading(false);

}

};

return (

<div className="auth-page">

  <div className="auth-card">

    { /* Icon placeholder — swap the dot for an <img> or SVG logo later */ }

    <div className="auth-icon">

      <div className="auth-icon-dot" />

    </div>

    <h1 className="auth-title">Log in to your account</h1>

    <p className="auth-subtitle">Welcome back! Please enter your details.</p>

    { /* Tab switcher: "Log in" is active here since this IS the login page.

      Clicking "Sign up" navigates to the Register page via <Link>,

      which does a client-side route change (no full page reload). */ }

    <div className="auth-tabs">

      <Link to="/register" className="auth-tab">Sign up</Link>

      <Link to="/login" className="auth-tab active">Log in</Link>

    </div>

    {error && <div className="auth-error">{error}</div>}

    <form onSubmit={handleSubmit}>

      <div className="auth-field">

        <label className="auth-label">Email</label>

```

```
<input
  className="auth-input"
  type="email"
  name="email"
  placeholder="Enter your email"
  value={form.email}
  onChange={handleChange}
  required
/>
</div>
```

```
<div className="auth-field">
  <label className="auth-label">Password</label>

  {/* Wrapper needed so the eye button can sit INSIDE the input's
    right edge (position: relative/absolute pairing in the CSS). */}

  <div className="auth-password-wrapper">

    <input
      className="auth-input"
      type={showPassword ? "text" : "password"} // toggles visibility
      name="password"
      placeholder="Enter your password"
      value={form.password}
      onChange={handleChange}
      required
    />


```

```
<button
  type="button" // IMPORTANT: type="button", not "submit" —
  // otherwise clicking the eye icon would submit the form
  className="auth-eye-btn"
  onClick={() => setShowPassword(!showPassword)}
  aria-label={showPassword ? "Hide password" : "Show password"}
>

  {showPassword ? "👁️" : "👁️"}

  {/* Swap these emoji for a proper icon library (e.g. lucide-react)
    once you add one — kept as emoji here so this file has
```

```

        zero extra dependencies to install right now. */}

</button>

</div>

</div>

<div className="auth-row">

  <label className="auth-remember">

    <input

      type="checkbox"

      checked={remember}

      onChange={(e) => setRemember(e.target.checked)}

    />

    Remember for 30 days

  </label>

  {/* TODO: not wired to a real backend route yet — there's no

    /api/forgot-password endpoint in your FastAPI app. This is

    a placeholder so the design matches; build the real flow

    (email a reset link, etc.) as its own future feature. */}

  <button type="button" className="auth-forgot-link">

    Forgot password

  </button>

</div>

<button type="submit" className="auth-submit-btn" disabled={loading}>

  {loading ? "Signing in..." : "Sign in"}

</button>

</form>

{/* TODO: cosmetic only for now — Google OAuth isn't implemented in

  your FastAPI backend (routers/auth.py only has email/password).

  Wiring this up for real means adding an OAuth flow server-side

  (e.g. Authlib) — a good candidate for a future sprint feature,

  not something to fake-connect to your current /api/login. */}

<button type="button" className="auth-google-btn">

```

```

    <span><img alt="Google logo" data-bbox="208 93 223 103"/> Sign in with Google
  </button>

  <p className="auth-bottom-text">
    Don't have an account? <Link to="/register">Sign up</Link>
  </p>
</div>
</div>
);
}

```

3. src/components/Register.jsx (rewritten, same shell)

```

jsx

import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { registerUser } from "../api";
import "../styles/auth.css";

export default function Register({ onSuccess }) {
  const [form, setForm] = useState({
    full_name: "",
    email: "",
    password: "",
    role: "job_seeker",
  });

  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const navigate = useNavigate();

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

```

```

setError("");

setLoading(true);

try {

  const data = await registerUser(form);

  // No "remember me" on registration in the screenshot's flow — a brand
  // new account just goes straight into a normal persisted session.

  localStorage.setItem("token", data.token);

  onSuccess && onSuccess(data.user);

  navigate("/dashboard");

} catch (err) {

  setError(err.message);

} finally {

  setLoading(false);

}

};

return (

<div className="auth-page">

  <div className="auth-card">

    <div className="auth-icon">

      <div className="auth-icon-dot" />

    </div>

    <h1 className="auth-title">Create your account</h1>

    <p className="auth-subtitle">Join CareerConnect to get started.</p>

    { /* Same tab switcher, but "Sign up" is active here instead. */ }

    <div className="auth-tabs">

      <Link to="/register" className="auth-tab active">Sign up</Link>

      <Link to="/login" className="auth-tab">Log in</Link>

    </div>

    {error && <div className="auth-error">{error}</div>}

    <form onSubmit={handleSubmit}>

```

```
<div className="auth-field">

  <label className="auth-label">Full name</label>

  <input

    className="auth-input"

    type="text"

    name="full_name"

    placeholder="Enter your full name"

    value={form.full_name}

    onChange={handleChange}

    required

  />

</div>
```

```
<div className="auth-field">

  <label className="auth-label">Email</label>

  <input

    className="auth-input"

    type="email"

    name="email"

    placeholder="Enter your email"

    value={form.email}

    onChange={handleChange}

    required

  />

</div>
```

```
<div className="auth-field">

  <label className="auth-label">Password</label>

  <div className="auth-password-wrapper">

    <input

      className="auth-input"

      type={showPassword ? "text" : "password"}

      name="password"

      placeholder="At least 8 characters"

      value={form.password}

    />

  </div>

</div>
```

```
onChange={handleChange}
```

```
required
```

```
minLength={8} // matches backend's Field(min_length=8) in schemas/auth.py —
```

```
// this just gives instant browser-side feedback; the real
```

```
// enforcement still happens server-side, don't remove that.
```

```
/>
```

```
<button
```

```
type="button"
```

```
className="auth-eye-btn"
```

```
onClick={() => setShowPassword(!showPassword)}
```

```
aria-label={showPassword ? "Hide password" : "Show password"}
```

```
>
```

```
{showPassword ? "👁️" : "🙈"}
```

```
</button>
```

```
</div>
```

```
</div>
```

```
<div className="auth-field">
```

```
<label className="auth-label">I am a</label>
```

```
{/* Kept as a plain <select> — doesn't fit the pill/tab visual
```

```
style from the screenshot, but a role picker wasn't part of
```

```
the reference design at all. If you want it to visually
```

```
match, this could become a second small tab-pair instead —
```

```
let me know if you want that version. */}
```

```
<select
```

```
className="auth-input"
```

```
name="role"
```

```
value={form.role}
```

```
onChange={handleChange}
```

```
>
```

```
<option value="job_seeker">Job Seeker</option>
```

```
<option value="recruiter">Recruiter</option>
```

```
</select>
```

```
</div>
```

```
<button type="submit" className="auth-submit-btn" disabled={loading}>
  {loading ? "Creating account..." : "Sign up"}
</button>
</form>

{/* Same TODO as Login.jsx — cosmetic placeholder, not wired up. */}
<button type="button" className="auth-google-btn">
  <span><img alt="Google logo" data-bbox="208 275 225 288"/></span> Sign up with Google
</button>

<p className="auth-bottom-text">
  Already have an account? <Link to="/login">Log in</Link>
</p>
</div>
</div>
);
}
```